Arturito — código versión 1 (respaldo)

Robot seguidor de línea · Gaby Leryn Ayala Chambi, Christopher Darius Valero Lima y Maximiliano Sosa Useche · U.E. Jaime Alfonso Escalante Gutierrez A, Santa Cruz de la Sierra · 2026

Primera versión del firmware, guardada como referencia. Usa el mismo cableado que la versión 2 (sensores en A0–A4, driver en D5/D6/D9/D10, EEP en D4), documentado en el PDF principal.

Convierte cada lectura a blanco o negro con un umbral (500) y decide el movimiento con una tabla de casos. Al encender, despierta el driver y corre una prueba de motores (adelante, pausa, atrás) antes de empezar a seguir la línea.

```
/*
* ============================================================
* ROBOT: ARTURITO - SEGUIDOR DE LÍNEA AMATEUR (CON PRUEBA DE MOTORES)
* CONCURSO: 1er Concurso Distrital de Robótica 2026 (Santa Cruz)
* ============================================================
* Hardware:
* - Arduino UNO
* - 5 Sensores TCRT5000 (Analógicos en A0..A4)
* - Driver DRV8833 (Pines D5, D6, D9, D10)
* - 2 Motores HC02-48 (1:48)
*
* CALIBRACIÓN:
* Blanco: 30 - 40
* Negro:  970 - 990
* Umbral: 500 (Negro > 500)
*/

// --- DEFINICIÓN DE PINES ---
// Sensores (Entradas Analógicas)
#define S0 A0 // Extremo Izquierdo
#define S1 A1 // Centro Izquierdo
#define S2 A2 // Centro
#define S3 A3 // Centro Derecho
#define S4 A4 // Extremo Derecho
```

```
// Driver DRV8833 (Salidas PWM)
#define IN1 5  // Motor Izquierdo Adelante
#define IN2 6  // Motor Izquierdo Atrás
#define IN3 9  // Motor Derecho Adelante
#define IN4 10 // Motor Derecho Atrás
// EEP (sleep) del modulo a D4: HIGH = driver habilitado.
// Sin este cable el chip duerme y las salidas no conducen.
#define SLEEP_DRV 4

// --- PARÁMETROS DE VELOCIDAD Y CALIBRACIÓN ---
const int VEL_RAPIDA = 180;
const int VEL_MEDIA  = 150;
const int VEL_LENTA  = 120;

// Umbral ajustado según tus lecturas
const int UMBRAL = 500;

// Variable para recordar la última dirección conocida
int ultimoGiro = 0; // 0 = recto, 1 = izquierda, 2 = derecha
```

```
void setup() {
// Configurar pines del driver como salidas
pinMode(IN1, OUTPUT);
pinMode(IN2, OUTPUT);
pinMode(IN3, OUTPUT);
pinMode(IN4, OUTPUT);

// Despertar el DRV8833 (EEP en HIGH) antes de tocar motores
pinMode(SLEEP_DRV, OUTPUT);
digitalWrite(SLEEP_DRV, HIGH);

// Iniciar comunicación serial para depurar
Serial.begin(9600);

// Pausa de seguridad inicial (2 segundos)
delay(2000);

// ========================================
// PRUEBA DE MOTORES (Adelante y Retorno)
// ========================================
// 1. Avanzar un poco (800 milisegundos a velocidad media)
avanzar(VEL_MEDIA, VEL_MEDIA);
delay(800);
```

```
// 2. Detenerse brevemente
detener();
delay(300);

// 3. Retroceder exactamente el mismo tiempo para volver al punto inicial
retroceder(VEL_MEDIA, VEL_MEDIA);
delay(800);

// 4. Detenerse antes de arrancar con el seguidor de línea
detener();
delay(500);
}

void loop() {
// 1. LEER SENSORES (LOGICA INVERTIDA: > UMBRAL es Negro)
bool v0 = analogRead(S0) > UMBRAL;
bool v1 = analogRead(S1) > UMBRAL;
bool v2 = analogRead(S2) > UMBRAL;
bool v3 = analogRead(S3) > UMBRAL;
bool v4 = analogRead(S4) > UMBRAL;

// 2. LÓGICA DE MOVIMIENTO

// --- CASO 1: CENTRADO PERFECTO ---
```

```
if (v2 && !v0 && !v1 && !v3 && !v4) {
avanzar(VEL_RAPIDA, VEL_RAPIDA);
ultimoGiro = 0;
}
// --- CASO 2: DESVIADO LEVEMENTE A LA IZQUIERDA ---
else if (v3 || (v2 && v3)) {
avanzar(VEL_RAPIDA, VEL_LENTA);
ultimoGiro = 2;
}
// --- CASO 3: DESVIADO LEVEMENTE A LA DERECHA ---
else if (v1 || (v2 && v1)) {
avanzar(VEL_LENTA, VEL_RAPIDA);
ultimoGiro = 1;
}
// --- CASO 4: CURVA CERRADA A LA IZQUIERDA ---
else if (v4 || (v3 && v4)) {
girarDerecha();
ultimoGiro = 2;
}
// --- CASO 5: CURVA CERRADA A LA DERECHA ---
else if (v0 || (v0 && v1)) {
girarIzquierda();
ultimoGiro = 1;
}
// --- CASO 6: INICIO DE CURVA SUAVE ---
```

```
else if (v0 && !v1 && !v2) {
girarIzquierda();
ultimoGiro = 1;
}
else if (v4 && !v3 && !v2) {
girarDerecha();
ultimoGiro = 2;
}
// --- CASO 7: CRUCE ---
else if (v0 && v1 && v2 && v3 && v4) {
avanzar(VEL_RAPIDA, VEL_RAPIDA);
}
// --- CASO 8: LÍNEA PERDIDA ---
else if (!v0 && !v1 && !v2 && !v3 && !v4) {
if (ultimoGiro == 1) {
girarIzquierda();
} else if (ultimoGiro == 2) {
girarDerecha();
} else {
detener();
}
}
// --- CASO POR DEFECTO ---
```

```
else {
avanzar(VEL_MEDIA, VEL_MEDIA);
}

delay(5);
}

// ========================================
// FUNCIONES DE MOVIMIENTO
// ========================================

void avanzar(int velIzq, int velDer) {
analogWrite(IN1, velIzq);
digitalWrite(IN2, LOW);
analogWrite(IN3, velDer);
digitalWrite(IN4, LOW);
}

void retroceder(int velIzq, int velDer) {
digitalWrite(IN1, LOW);
analogWrite(IN2, velIzq);
digitalWrite(IN3, LOW);
analogWrite(IN4, velDer);
}
```

```
void girarIzquierda() {
analogWrite(IN1, 0);
digitalWrite(IN2, LOW);
analogWrite(IN3, VEL_RAPIDA);
digitalWrite(IN4, LOW);
}

void girarDerecha() {
analogWrite(IN1, VEL_RAPIDA);
digitalWrite(IN2, LOW);
analogWrite(IN3, 0);
digitalWrite(IN4, LOW);
}

void detener() {
digitalWrite(IN1, LOW);
digitalWrite(IN2, LOW);
digitalWrite(IN3, LOW);
digitalWrite(IN4, LOW);
}
```